

Strategy for If/Else

- Test condition
 - Jump to else block whenever the condition is false
 - Otherwise, perform the if block and jump to the code after
 - Flowcharts can be helpful
-

Example

Translate the following C code into RISC-V assembly:

```
if ( i == j ) {  
    i++;  
} else {  
    j-;  
}  
  
j += i;
```

```
# assume i in t0, j in t1  
  
# condition check  
bne t0, t1, else  
  
# if block  
addi t0, t0, 1  
j end  
  
# else block  
else:  
    addi t1, t1, -1  
  
# rest of code  
li a7, 10  
ecall
```

Example

Translate the following code to RISC-V assembly

```
int i = 0;
while (i != 100) {
    i++;
}
```

```
li t0, 0

# Condition check
LOOP_CONDITION:
    li t1, 100
    beq t0, t1, END

# Loop code
LOOP:
    addi t0, t0, 1
    j LOOP_CONDITION

# Post-loop code
END:
    li a7, 10
    ecall
```

Interacting with Memory

- All of the code examples thus far perform operations on *registers* and *immediate values*
 - How can we perform operations on memory?
- All programs must fetch values from memory into registers, operate on them, and then store the values back into memory
 - Register-to-register architecture
- Memory operations have the form
 - `lw t0, 12(s0)`
 - `lw` → Load or store
 - `t0` → Local data register
 - `12` → Offset from memory address
 - `(s0)` → Register storing base address of data value in memory
 - Roughly equal to: `t0 = s0[3]`
- We need a memory model to understand what these operations mean
 - CSC148 and CSC207 use box and arrow diagrams
 - CSC258 and CSC209 use a model of the entire memory space

How are values stored in memory?

- 32-bit integers → 4 consecutive bytes
 - Int arrays → Consecutive 4-byte chunks
 - 8-bit characters → A single byte
 - C Strings: Consecutive bytes, terminated by NULL
 - Instructions: Stored in 4 bytes (for 32-bit ISAs)
-

Data storage in assembly

- Always of the form “label size value”
- Memory is not typed
 - Registers and memory only store binary values
- Up to the developer to ascribe meaning to the binary values

```
.data
var1: .word 3                # Single integer with initial value
of 3
array0: .word 3, 7, 5, 42    # 4-element integer array
array1: .byte 97, 98        # 2-element character array with
elements 'a', 'b'
.align 4
array2: .zero 40            # Allocate 40 consecutive bytes that are
zeroed out
```

Function Calls

- Functions allow us to structure and reuse code
- Functions are governed by conventions that were affected by hardware limitations
 - Only a single value is returned
 - Arguments may be passed
 - Local variables do not escape the environment
- Constrained by the compiler
 - Firm rules about how to transfer control to and from functions
- Every function call needs its own distinct set of local variables, and these variables disappear when the function ends
 - Located in a stack frame
- Functions must be invoked on arguments
 - Argument values must be transferred
- Every function call needs to return to the location that invoked it
 - Store a return address and return value

Accessing the Stack

- The address of the top of the stack is stored in register **sp**
- Pushing value in **t0** into the stack:

```
addi sp, sp, -4 # Move stack pointer to make space
sw t0, 0(sp) # Push a word onto the stack
```

- Pop a value from the stack and store it in **t0**

```
lw t0, 0(sp) # Pop a word from the stack
addi sp, sp, 4 # Update stack pointer, stack size smaller
```

Function Call Costs

- We based on one function call convention that we defined
 - There could be other conventions
- Function calls don't happen for free
 - Lots of register moves
 - Data may need to be stored in memory
 - Jumps are not quick instructions

Syscalls

- Function calls to the operating system
- Have a convention for how arguments are sent
 - **a0**, **a1**, and **a2** registers are used for arguments
 - **a7** stores the name of the function (syscall to be invoked)
- The `ecall` instruction invokes a handler that deals with the request
- Return values (if any) are often placed directly on the stack